

Φάση A: Data Preprocessing

Υπεύθυνος: Data Engineer

Ακολουθείται το μοτίβο **Bronze** → **Silver** → **Gold** για σταδιακή μετατροπή των ακατέργαστων δεδομένων σε έτοιμα προς μοντελοποίηση feature vectors. Σε κάθε στάδιο τα δεδομένα αποκτούν μεγαλύτερη δομή και καθαρότητα:

- **Bronze:** Ακατέργαστα δεδομένα, φόρτωση από CSV, αφαίρεση περιττών στηλών, καθαρισμός τιμών.
- **Silver:** Καθαρισμένα και κωδικοποιημένα, οι κατηγορικές μεταβλητές αποκτούν αριθμητικούς δείκτες, οι κενές τιμές συμπληρώνονται.
- **Gold:** Έτοιμα για μοντελοποίηση, one-hot encoding, κανονικοποίηση, oversampling.

Όλες οι οπτικοποιήσεις έχουν μεταφερθεί στο EDA notebook.

Βήματα:

1. Bronze Layer, φόρτωση και καθαρισμός
2. Train / Test split
3. Imputation, συμπλήρωση κενών τιμών
4. Silver Layer, categorical encoding
5. SMOTE, αντιμετώπιση class imbalance
6. Gold Layer, one-hot encoding + scaling
7. Αποθήκευση για επόμενα notebooks

1. Εκκίνηση του περιβάλλοντος επεξεργασίας

Η επεξεργασία εκτελείται σε τοπικό περιβάλλον αξιοποιώντας όλους τους διαθέσιμους πυρήνες του μηχανήματος, δεν απαιτείται cluster. Δημιουργείται ο φάκελος data/ όπου θα αποθηκευτούν όλα τα ενδιάμεσα και τελικά αποτελέσματα της προεπεξεργασίας. Η χρήση ενός κεντρικού φακέλου διασφαλίζει ότι τα επόμενα notebooks μπορούν να φορτώσουν τα δεδομένα απ' ευθείας, χωρίς να ξανατρέξουν ολόκληρη τη διαδικασία.

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, lit
```

```
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.family'] = 'DejaVu Sans'
```

```
spark = SparkSession.builder \
    .appName("Stroke_DataEngineering") \
    .master("local[*]") \
    .getOrCreate()
```

```
os.makedirs("../data", exist_ok=True)

print("SparkSession αρχικοποιήθηκε")

WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/06/01 18:57:01 WARN Utils: Your hostname, cacyos-x8664, resolves to a
loopback address: 127.0.1.1; using 192.168.1.5 instead (on interface enp4s0)
26/06/01 18:57:01 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
26/06/01 18:57:02 WARN NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable

SparkSession αρχικοποιήθηκε
```

2. Bronze Layer, πρώτη επαφή με τα ακατέργαστα δεδομένα

Το dataset φορτώνεται από το αρχικό CSV. Η στήλη `id` αφαιρείται αμέσως, είναι τεχνικό αναγνωριστικό χωρίς καμία προγνωστική αξία. Η διατήρησή της θα εισήγαγε θόρυβο και θα μπορούσε να οδηγήσει σε υπερπροσαρμογή.

Η στήλη `bmi` παρουσιάζει ένα ιδιαίτερο πρόβλημα: περιέχει την τιμή "N/A" ως string αντί για κενό (`null`). Αυτό σημαίνει ότι κατά την αυτόματη ανίχνευση σχήματος ολόκληρη η στήλη θα αναγνωριστεί ως κείμενο αντί για αριθμητική. Το πρόβλημα διορθώνεται στο επόμενο βήμα με ρητή μετατροπή.

```
df_raw = spark.read.csv("../healthcare-dataset-stroke-data.csv", header=True,
inferSchema=True)
df = df_raw.drop("id")

bronze_count = df.count()
print(f"Bronze Layer – {bronze_count} γραμμές, {len(df.columns)} στήλες")
```

Bronze Layer – 5110 γραμμές, 11 στήλες

3. Καθαρισμός του BMI

Το string "N/A" αντικαθίσταται με πραγματική κενή τιμή (`null`) και η στήλη μετατρέπεται σε αριθμητική μορφή. Από τις 5.110 εγγραφές, οι 201 (3.9%) είχαν ελλιπές BMI.

Κρίσιμη σχεδιαστική επιλογή: η συμπλήρωση των κενών τιμών **δεν** γίνεται τώρα. Αν συμπληρώναμε τις τιμές πριν τον διαχωρισμό `train/test`, θα υπήρχε διαρροή πληροφορίας (data leakage), το `test set` θα «έβλεπε» έμμεσα τα στατιστικά του `train`. Το `imputation` θα γίνει αφού το dataset χωριστεί, χρησιμοποιώντας μόνο τα δεδομένα εκπαίδευσης.

```
from pyspark.sql.types import DoubleType

df = df.withColumn(
    "bmi",
```

```

    when(col("bmi") == "N/A", lit(None)).otherwise(col("bmi")).cast(DoubleType())
)

null_bmi_count = df.filter(col("bmi").isNull()).count()
print(f"Nulls στο bmi: {null_bmi_count} ({(null_bmi_count / bronze_count) *
100:.1f}%)")

Nulls στο bmi: 201 (3.9%)

```

4. Διαχωρισμός σε σύνολα εκπαίδευσης και ελέγχου

Ο διαχωρισμός γίνεται **πριν από οποιονδήποτε μετασχηματισμό**, με αναλογία 80/20. Αυτό είναι θεμελιώδες για την ακεραιότητα του πειράματος: το test set λειτουργεί ως προσομοίωση άγνωστων δεδομένων που θα συναντήσει το μοντέλο στην παραγωγή. Κανένα στατιστικό μέτρο, μετασχηματισμός ή παράμετρος δεν υπολογίζεται από το test, όλα προέρχονται αποκλειστικά από το train.

Η χρήση σταθερού σπόρου τυχαιότητας (seed=42) διασφαλίζει ότι ο διαχωρισμός είναι αναπαραγωγίσιμος, κάθε εκτέλεση του notebook παράγει τα ίδια ακριβώς σύνολα.

```

train, test = df.randomSplit([0.8, 0.2], seed=42)

train_count = train.count()
test_count = test.count()
print(f"Train: {train_count} | Test: {test_count}")

Train: 4123 | Test: 987

```

5. Συμπλήρωση ελλειπόν τιμών BMI

Οι κενές τιμές του BMI συμπληρώνονται με τη διάμεσο (median), η οποία υπολογίζεται αποκλειστικά από το train set. Η επιλογή της διαμέσου έναντι του μέσου όρου είναι σκόπιμη: η διάμεσος είναι ανθεκτική σε ακραίες τιμές (outliers) και παραμένει αντιπροσωπευτική ακόμα κι αν η κατανομή είναι ασύμμετρη.

Το test set λαμβάνει την ίδια τιμή διαμέσου, η παράμετρος μεταφέρεται από το train χωρίς να επανυπολογιστεί, αποφεύγοντας έτσι τη διαρροή πληροφορίας.

```

from pyspark.ml.feature import Imputer

imputer = Imputer(inputCols=["bmi"], outputCols=["bmi_imputed"],
strategy="median")
imputer_model = imputer.fit(train)

train =
imputer_model.transform(train).drop("bmi").withColumnRenamed("bmi_imputed",
"bmi")
test = imputer_model.transform(test).drop("bmi").withColumnRenamed("bmi_imputed",
"bmi")

median_val = imputer_model.surrogateDF.collect()[0][0]
print(f"Median BMI: {median_val:.2f}")
print(f"Nulls after imputation - train:

```

```
{train.filter(col('bmi').isNull()).count()}, test:
{test.filter(col('bmi').isNull()).count()}"
```

Median BMI: 27.90

Nulls after imputation - train: 0, test: 0

6. Silver Layer, κωδικοποίηση κατηγορικών μεταβλητών

Οι αλγόριθμοι μηχανικής μάθησης απαιτούν αριθμητική είσοδο. Οι πέντε κατηγορικές μεταβλητές (φύλο, οικογενειακή κατάσταση, τύπος εργασίας, τύπος περιοχής, κατάσταση καπνίσματος) μετατρέπονται σε αριθμητικούς δείκτες βάσει της συχνότητας εμφάνισής τους, η πιο συχνή κατηγορία λαμβάνει τον δείκτη 0, η επόμενη τον 1, κ.ο.κ.

Οι αρχικές στήλες με τα string labels διατηρούνται παράλληλα με τους δείκτες. Αυτό επιτρέπει στο EDA notebook να παράγει ευανάγνωστες οπτικοποιήσεις χρησιμοποιώντας τα ονόματα των κατηγοριών αντί για αριθμούς.

Προβλέπεται ειδική μεταχείριση για άγνωστες κατηγορίες: αν το test set περιέχει μια τιμή που δεν εμφανίστηκε στο train, αυτή αντιστοιχίζεται σε ξεχωριστή θέση αντί να απορριφθεί.

```
from pyspark.ml.feature import StringIndexer
from pyspark.ml import Pipeline
```

```
cat_cols = ["gender", "ever_married", "work_type", "Residence_type",
"smoking_status"]
```

```
indexers = [
    StringIndexer(inputCol=c, outputCol=f"{c}_index", handleInvalid="keep")
    for c in cat_cols
]
```

```
idx_pipeline = Pipeline(stages=indexers)
idx_model = idx_pipeline.fit(train)
```

```
train = idx_model.transform(train)
test = idx_model.transform(test)
```

```
# Αφαίρεση metadata για συμβατότητα με OneHotEncoder
```

```
for c in cat_cols:
    ic = f"{c}_index"
    train = train.withColumn(ic, col(ic))
    test = test.withColumn(ic, col(ic))
```

```
print("StringIndexer ολοκληρώθηκε")
```

StringIndexer ολοκληρώθηκε

Αποθήκευση Silver Layer

Σε αυτό το σημείο τα δεδομένα είναι πλήρως καθαρισμένα και κωδικοποιημένα: όλες οι κενές τιμές έχουν συμπληρωθεί, οι κατηγορικές μεταβλητές έχουν αριθμητικούς δείκτες, και οι αρχικές

ετικέτες διατηρούνται για ευανάγνωστη ανάλυση. Το Silver Layer αποθηκεύεται σε Parquet ώστε το EDA notebook να έχει άμεση πρόσβαση χωρίς να ξανατρέχει τα προηγούμενα βήματα.

```
train.write.mode("overwrite").parquet("../data/train_silver.parquet")
test.write.mode("overwrite").parquet("../data/test_silver.parquet")

print(f"Silver Layer αποθηκεύτηκε - train: {train.count()} γραμμές, test:
{test.count()} γραμμές")
```

Silver Layer αποθηκεύτηκε - train: 4123 γραμμές, test: 987 γραμμές

7. SMOTE, εξισορρόπηση των κλάσεων

Το dataset πάσχει από έντονη ανισορροπία: μόλις το ~5% των ασθενών έχει υποστεί εγκεφαλικό. Ένα μοντέλο που θα προέβλεπε πάντα «όχι stroke» θα πετύχαινε 95% accuracy, αλλά θα ήταν εντελώς άχρηστο στην πράξη.

Το SMOTE (Synthetic Minority Oversampling Technique) δημιουργεί συνθετικά παραδείγματα της μειοψηφικής κλάσης. Για κάθε πραγματικό ασθενή με stroke, ο αλγόριθμος επιλέγει έναν κοντινό γείτονα στον χώρο των χαρακτηριστικών και παράγει ένα νέο σημείο στην ευθεία που τους ενώνει. Το αποτέλεσμα είναι ένα ισορροπημένο train set με ίσο αριθμό παραδειγμάτων ανά κλάση.

Η διαδικασία εφαρμόζεται **αποκλειστικά** στο train set. Το test παραμένει ανέγγιχτο με τη φυσική του κατανομή, ώστε η αξιολόγηση να αντικατοπτρίζει πραγματικές συνθήκες. Οι συνθετικοί δείκτες κατηγοριών στρογγυλοποιούνται και περιορίζονται στα έγκυρα όρια, καθώς η παρεμβολή μπορεί να παράγει δεκαδικές τιμές όπου π.χ. ο δείκτης φύλου 1.3 δεν έχει νόημα και στρογγυλοποιείται σε 1.

```
from imblearn.over_sampling import SMOTE

numeric_cols = ["age", "hypertension", "heart_disease", "avg_glucose_level",
                "bmi"]
indexed_cols = [f"{c}_index" for c in cat_cols]
features_smote = numeric_cols + indexed_cols

# Μέγιστο έγκυρο index ανά στήλη
idx_stages = idx_model.stages
max_idx = {f"{cat_cols[i]}_index": len(idx_stages[i].labels) - 1 for i in
           range(len(cat_cols))}

train_pdf = train.select(features_smote + ["stroke"]).toPandas()

X_res, y_res = SMOTE(random_state=42).fit_resample(train_pdf[features_smote],
                                                    train_pdf["stroke"])

bal_pdf = pd.DataFrame(X_res, columns=features_smote)
bal_pdf["stroke"] = y_res

for c in indexed_cols:
    bal_pdf[c] = bal_pdf[c].round().clip(0, max_idx[c])
```

```
train_bal = spark.createDataFrame(bal_pdf)

bal_total = train_bal.count()
print(f"Train μετά από SMOTE: {bal_total} δείγματα ({bal_total // 2} ανά κλάση)")

Train μετά από SMOTE: 7832 δείγματα (3916 ανά κλάση)
```

8. Gold Layer, τελική προετοιμασία για μοντελοποίηση

Το Gold Layer είναι το τελικό στάδιο. Εδώ ολοκληρώνονται τρεις κρίσιμοι μετασχηματισμοί:

One-hot encoding: Οι αριθμητικοί δείκτες των κατηγορικών μεταβλητών μετατρέπονται σε δυαδικές στήλες, μία ανά κατηγορία. Χωρίς one-hot encoding, το μοντέλο μπορεί να θεωρήσει λανθασμένα ότι η κατηγορία με δείκτη 3 είναι «μεγαλύτερη» από αυτήν με δείκτη 1 (ordinal bias). Με το one-hot, κάθε κατηγορία είναι ανεξάρτητη δυαδική στήλη.

Συνένωση: Όλες οι δυαδικές και αριθμητικές στήλες συγκεντρώνονται σε ένα ενιαίο διάνυσμα χαρακτηριστικών, το feature vector που θα τροφοδοτήσει τα μοντέλα.

Κανονικοποίηση: Κάθε διάσταση του διανύσματος διαιρείται με την τυπική της απόκλιση. Τα χαρακτηριστικά αποκτούν συγκρίσιμη κλίμακα, απαραίτητο για αλγόριθμους που βασίζονται σε αποστάσεις (π.χ. logistic regression, SVM).

Όλες οι παράμετροι (κατηγορίες one-hot, τυπικές αποκλίσεις) υπολογίζονται αποκλειστικά από το ισορροπημένο train set και εφαρμόζονται και στα δύο σύνολα.

```
from pyspark.sql.types import DoubleType
from pyspark.ml.feature import OneHotEncoder, VectorAssembler, StandardScaler

# Αφαίρεση metadata από το test για συμβατότητα
test = test.select(
    *[col(c) for c in test.columns if not c.endswith("_index")],
    *[col(c).cast(DoubleType()).alias(c) for c in test.columns if
c.endswith("_index")]
)

encoders = [
    OneHotEncoder(inputCol=f"{c}_index", outputCol=f"{c}_vec",
handleInvalid="keep")
    for c in cat_cols
]

assembler = VectorAssembler(
    inputCols=[f"{c}_vec" for c in cat_cols] + numeric_cols,
    outputCol="assembled"
)

scaler = StandardScaler(inputCol="assembled", outputCol="features", withStd=True,
withMean=False)
```

```
gold_pipeline = Pipeline(stages=encoders + [assembler, scaler])
gold_model = gold_pipeline.fit(train_bal)

train_gold = gold_model.transform(train_bal).select("features", "stroke")
test_gold = gold_model.transform(test).select("features", "stroke")

print(f"Train Gold: {train_gold.count()} samples")
print(f"Test Gold: {test_gold.count()} samples")
```

```
Train Gold: 7832 samples
Test Gold: 987 samples
```

9. Αποθήκευση του Gold Layer

Το τελικό αποτέλεσμα αποθηκεύεται σε Parquet, μια συμπιεσμένη, columnar μορφή αρχείου βελτιστοποιημένη για το Spark. Το Parquet διατηρεί το σχήμα των δεδομένων, επιτρέπει γρήγορη ανάγνωση επιλεγμένων στηλών και μειώνει σημαντικά τον απαιτούμενο χώρο στον δίσκο. Τα επόμενα notebooks φορτώνουν απ' ευθείας από αυτά τα αρχεία χωρίς να ξανατρέχουν ολόκληρη την προεπεξεργασία.

```
train_gold.write.mode("overwrite").parquet("../data/train_gold.parquet")
test_gold.write.mode("overwrite").parquet("../data/test_gold.parquet")
```

```
print("Αποθηκεύτηκαν")
```

```
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 95.00% for 8 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 84.44% for 9 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 76.00% for 10 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 69.09% for 11 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 63.33% for 12 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 58.46% for 13 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 54.29% for 14 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 58.46% for 13 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
```

Scaling row group sizes to 63.33% for 12 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 69.09% for 11 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 76.00% for 10 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 84.44% for 9 writers
26/06/01 18:57:13 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 95.00% for 8 writers

Αποθηκεύτηκαν